

038 容量与插值实验指南

本实验目标是判断一条容量曲线究竟支持什么结论。先复核同一两样本的完整训练链，再改变噪声和线性特征数量，分开记录训练拟合、真函数风险、谱病态和正则化。不要预设每张图都必须出现双下降，也不要没有出现预期形状的运行删掉。

1 环境与数据

实际核验环境为Linux CPU, Python3.12.14、NumPy2.3.5、PyTorch2.7.1+cpu、Matplotlib3.10.8, float64。无需GPU、在线数据、账号或付费API。使用本讲environment.yml建立独立环境，或在已有Python3.12环境依README安装。环境声明并非全新Anaconda跨平台安装已验证。

在本讲目录执行：

```
python experiment.py --output my-run
python -m unittest -v test_experiment
jupyter lab
```

Notebook先重启内核，再顺序运行。首格会检查代码、数据和图的哈希；不要跳过首格并阅读旧输出。制作时采用全新Python进程中的真实IPython InProcessKernel，未测试浏览器界面或外进程socket传输。输出目录可换名字；脚本用自身位置定位数据，不依赖启动时工作目录。

数据是固定合成fixture：12组 24×96 的Gaussian训练输入、每组24个标准正态噪声，seed3800至3811。宽度18种，噪声强度3种，惩罚3种，合计1944拟合。主实验直接读取fixture；独立测试按声明seed重新生成并逐元素比较。任何修改fixture会先失败，而不是把新数据与旧图混在一起。探索新设定时复制代码和数据到自己目录，建立新的记录，不冒充本讲基准结果。

2 手算任务

X两行分别为(1,0,1)、(0,1,1)，y为1、2。初始 $w=(0,0,0)$ ，平均半平方损失，学习率0.5。

1. 不运行代码，写出每行的三个乘积、预测、残差、单样本损失和平均目标。
2. 写出从平均号、半平方、残差、线性预测到每个参数的局部导数，填满 2×3 的贡献表。
3. 对每列累加，三个参数同时更新；重复第二步更新和第三次前向。
4. 检查第一次更新后已拟合的样本，为什么第二次更新后又出现误差？

完成后读取outputs/hand-trace.json。不要只核对最终loss；任一局部路径错误都可能被巧合抵消。预期损失链是1.25、0.140625、0.0439453125。

3 相同训练预测与不同未来预测

推导 $Xz=0$ 的全部非零方向。用 $w^*=(0,1,1)$ 和 $t=-1, 0, 1$ 构造全部插值解的一部分，分别计算训练预测、参数平方范数、新点 $(1,1,0)$ 预测。接着运行80步GD，确认零空间分量保持。

再令 $w=\text{diag}(1,1,2)u$ ，比较最小 u 范数解与最小 w 范数解。写出两个算法是否优化同样的训练平方损失、是否有同样的函数类、是否选择同样的解。这三问应分别回答。

可直接运行以下完整片段；其中`np.linalg.lstsq`是独立参考求解入口，不调用主脚本的SVD解。

```
import numpy as np
X = np.array([[1., 0., 1.], [0., 1., 1.]])
y = np.array([1., 2.])
w = np.linalg.lstsq(X, y, rcond=1e-12)[0]
S = np.diag([1., 1., 2.])
u = np.linalg.lstsq(X @ S, y, rcond=1e-12)[0]
print("ordinary", w, "rescaled", S @ u)
print("training predictions", X @ w, X @ S @ u)
print("new-point predictions", np.array([1., 1., 0.]) @ w,
      np.array([1., 1., 0.]) @ S @ u)
```

4 复制神经元不保证复制训练动力学

使用正文同一两样本，先算6参数网络，再复制隐藏节点并平分输出权重形成11参数网络。把所有参数按`duplication.json`的`parameter_order`排好。

对两个模型各自填写：每个样本 z 、ReLU门、输出、损失； $dL/d\text{prediction}$ 、 $d\text{prediction}/dh$ 、 dh/dz ； $2 \times P$ 的参数贡献；列和梯度；学习率0.2的同步更新；下一前向。样本1贡献为零仍需保留，不能把批次均值当成单样本目标。检验为什么输出偏置 c 只更新一次，复制节点的两个输出权重却各有自己的梯度。

先验证初始函数对任意输入相同，再解释为什么更新后可以不同。观察一步谁的训练损失更低，不推断长期泛化优劣。

5 阅读全部容量曲线

在`capacity.csv`中筛`ridge=0`，对每个 σ 、 p 保留12个seed。记录中位数、均值、范围和每次曲线。重点检查 $p=3, 22, 23, 24, 25, 26, 32, 48, 96$ ，不能跨过插值附近密集点只画稀疏宽度。

- 每个 p 的数值rank是否为 $\min(24, p)$?
- $\sigma=0$ 时，为什么 $3 \leq p \leq 24$ 的风险接近机器精度，而 $p > 24$ 后变大?
- $\sigma=0.8$ 时， $p=24$ 的中位数和均值差多少？哪次风险最大?
- $p=48$ 到96是否继续下降？报告实际结果。
- `ridge=0.01`和`0.1`分别怎样改变训练MSE及干净风险？固定惩罚不等于调参最优。

所有1944次系数、训练预测和标签保留在capacity-detail.json。不得声称代码记录了SVD算法内部迭代过程；这里是直接求解，不是1944条神经网络epoch轨迹。

6 核对风险而不是只看图

把拟合系数补零到96维，自己算与真 β 的平方距离。另将它拆为已建模参数误差和遗漏信号。核对它等于clean_risk，并说明为什么noisy_risk多出 σ^2 。

再核对signal_error、realized_noise_norm2、cross_term的和。对于Ridge或 $p < 3$ ，查看cross_term是否非零；不要把对噪声求期望后为零的量直接从单次样本中删除。

conditional_noise_mean_risk固定完整训练设计，仅平均新训练噪声；clean_risk固定这次训练标签，只平均新输入。

conditional-risk.json另外包含一个固定设计的400次噪声重复和一个固定拟合的4096新输入检查。标准误衡量Monte Carlo平均的波动，不是模型效果的置信保证，更不是12个主seed的标准误。

7 谱滤波与不符合直觉的结果

谱诊断使用 $X=\text{diag}(2,0.2)$ 、 $y=(1,1)$ 、 $\eta=0.4$ ，检查0、1、2、10、100、1000步。按正文等比数列式手算两个坐标，再对照spectral-filters.json。解释小奇异值方向为何学得慢，完全拟合后为何需要大系数。

比较Ridge谱因子时必须保留 $n\lambda$ ，不能把目标为平均损失的实现与总和损失公式混用。尝试反解使某一个方向的GD因子等于Ridge因子的 λ ，再检查另一个方向是否也匹配。

8 交付研究记录

提交一份记录，至少包含：明确问题和风险定义；固定数据/参数/种子清单；两轮手算与两种神经网络完整链；全部seed的容量图；一个无噪声反例、一个更宽反而更差反例；数值秩和谱说明；两条支持及两条不支持的结论；一个只改变单因素的后续实验计划。

最后写四句理论边界：通用逼近的量词和域；最小范数GD证明所需算法条件；有限类Hoeffding界的独立/有界/有限前提；比例渐近如何不同于 $n=24$ 实验。核对18组测试全部通过，再看详解，保留自己最初的错误及修正。